

Develop a Database

Internal Assessment Resource

Digital Technologies: Level 1

This resource supports assessment against Achievement Standard 92004

Standard title: Create a computer program

Credits: 5 Credits

Resource title: **My Database Application**

Authenticity of evidence	Assessor involvement during the assessment event is limited to providing general feedback which suggests sections of student work that would benefit from further development or skills a student may need to revisit across the work. Student work which has received sustained or detailed feedback is not suitable for submission towards this Standard.
--------------------------	---



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

Student/Ākonga instructions

Introduction

You are going to make a simple database application using Python and SQL in Visual Studio Code over four weeks. You will be using github for version control and to show incremental development.

This application can be your own design or a design can be given to you by your teacher. Please note that **databases should NOT be more than 3 tables**. A single table is acceptable as long as the design is correct and you may have to compromise to ensure the database does not become too complex.

[Appendix 1](#) contains a few examples.

Please ensure you have a sound grasp of SQL and Python before attempting this assessment. You must have completed all the relevant coursework before beginning this assessment as your teacher is NOT allowed to give you significant help beyond basic feedback during the assessment period.

You will be expected to be able to follow basic software engineering and programming conventions during this project.

Conventions include but are not limited to:

- Using an organised file structure for your project files
- Using well named files and folders in your project
- Has appropriate variable and function names that describe their purpose
- Using code comments where appropriate
- Following common language conventions for Python and SQL
- Following Database Design standards (eg normalized data to 3NF- well structured database)
- Using version control including descriptive commit messages

You are encouraged to incrementally develop your application through weekly iterations of “plan-develop-test”, keeping a record of your progress and testing as you go inside of this document. Git Version control should also be used as evidence of this.

If you are unable to use git, multiple files in your source code folder must be made with version number included. Failure to do so will limit your ability to gain a mark better than achieved.

You will submit the working application (stored on github or a zip of your entire source code folder) as well as this document for assessment against the criteria listed above.

Now complete the following questions.



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

Before Development

What is the purpose of this Application

This application allows users to manage cricket data including teams and players.
Users can add, view, search, update, and delete player records stored in a SQLite database.

Who might use this Application?

- Cricket fans who want to compare players and teams
- Coaches who want to track player performance data
- Students learning about databases and Python
- Sports analysts who want a simple way to store and retrieve cricket statistics

What are the requirements and specifications for this Application?

Requirements	Specifications
FUNCTIONAL REQUIREMENTS: - User must be able to add player records - User must be able to view all player records - User must be able to search players by team - User must be able to update player details - User must be able to delete player records NON-FUNCTIONAL REQUIREMENTS: - Program must run in Python using VS Code - Data must be stored in SQLite database - Program must use a menu-driven interface - Code must be structured using functions - Program must handle invalid inputs safely	CONSTRAINTS: - Maximum 2 tables (team, player) - No external database hosting required - No internet/API dependency SUCCESS CRITERIA: - All CRUD operations must work correctly - Database must maintain data integrity - User interface must be simple and readable



Database Design- Your Entity Relationship Diagram.

DATABASE DESIGN

TABLE 1: team

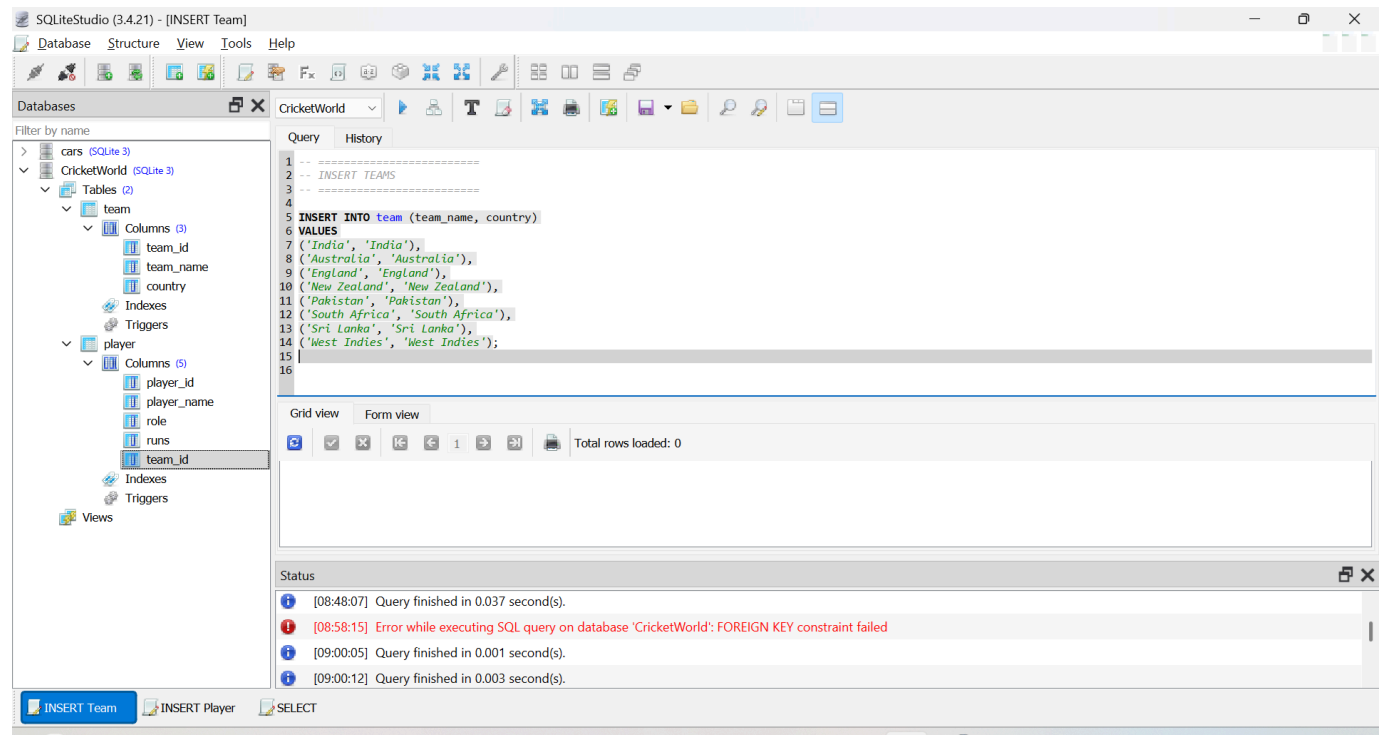
- team_id (INTEGER PRIMARY KEY)
- team_name (TEXT)
- country (TEXT)

TABLE 2: player

- player_id (INTEGER PRIMARY KEY)
- player_name (TEXT)
- role (TEXT)
- runs (INTEGER)
- team_id (INTEGER FOREIGN KEY -> team.team_id)

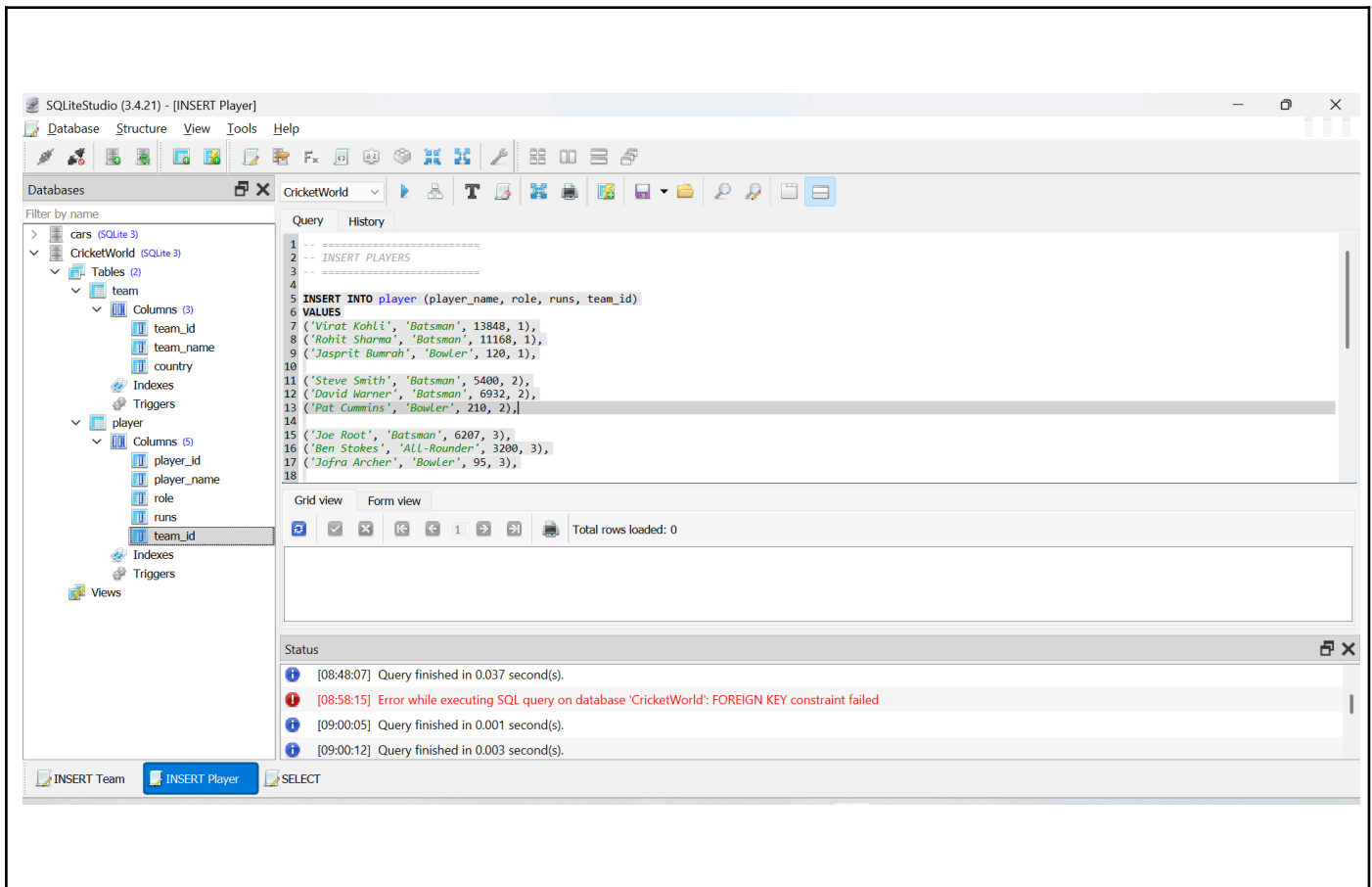
RELATIONSHIP:

- One team has many players (1-to-many relationship)



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz



In Development

Github repo

<https://github.com/Samarth-Agarwal24027/SQL-PYTHON/tree/main/Project-CricketWorld>

Steps to run my program:

1. Download the folder from my github:

<https://github.com/Samarth-Agarwal24027/SQL-PYTHON/tree/main/Project-CricketWorld>

2. Run this program:

https://github.com/Samarth-Agarwal24027/SQL-PYTHON/blob/main/Project-CricketWorld/development_CricketWorld.py

2.1 python3 development_CricketWorld.py

3. select options in the program to view the result



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

Database Testing Table: SQL Statements

Write down all the “purposes” that you think of for your database and then work out the SQL query that you might use. Test it and make sure it works here before moving on. Come back to this and add any more testing that you need as you expand your application.

Purpose	SQL Statement	Result Success?
Retrieve all players with team names	SELECT player.player_id, player.player_name, player.role, player.runs, team.team_name FROM player JOIN team ON player.team_id = team.team_id;	Yes
Search players by team name	SELECT player.player_name, player.role, player.runs FROM player JOIN team ON player.team_id = team.team_id WHERE team.team_name = 'New Zealand';	Yes
Insert New Player	INSERT INTO player (player_name, role, runs, team_id) VALUES ('Brendon McCullum', 'wicketkeeper batsman', 14676, 4);	Yes
Update New Runs	UPDATE player SET runs = 15000 WHERE player_id = 25;	Yes
Delete Player	DELETE FROM player WHERE player_id = 25;	Yes



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

Python Program Testing Table

Write down all the “inputs” that you can think of to test your program. Include a purpose too. What are these inputs supposed to test? Include the expected and actual results. Include unsuccessful tests and use this to help fix up your program. Test thoroughly to ensure your application is bulletproof and record all the test here before you hand in your application for marking.

Purpose	Input(s)	Expected Result(s)	Actual Result(s)
Add new player	Name: Brendon McCullum, Role: wicketkeeper batsman, Runs: 14676, Team ID: 4	Player added successfully	Player added successfully
View all players	Option 2	Display all players with team names	24+ players displayed correctly
Search Players by Team	Team: New Zealand	Show all NZ players including new player	Correct players displayed
Update player	ID: 25, Runs: 15000	Player runs updated	Player updated successfully
Delete player	ID: 25	Player removed from database	Player deleted successfully
Exit program	Option 6	Program Closes	Goodbye message shown

Project Complete

Once all testing is complete and you have ensured your github repo is up to date, you can hand your application in for marking.



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

Appendix 1

GPU Database

A database of modern graphics cards, their speeds and their prices for PC gamers who want to be able to compare cards by manufacturer, price and speed.

Sports Team Tracker

A sports team database for a coach to keep track of players attendance, games and points scored over the season.

Library Catalog Database

A comprehensive catalog system allowing librarians to manage book details, borrower information, and lending status.

Student Grade Tracker

An efficient tool enabling teachers to record and analyze student grades for assignments, quizzes, and tests.

Inventory Management System

A streamlined solution for businesses to monitor product inventory, suppliers, and reordering needs.

Study Guide

A program to help students study by asking quick quiz questions and checking their answers. They can customise it by being able to add or delete questions from the database.

Recipe Organizer

A user-friendly platform for culinary enthusiasts to collect and categorize recipes, along with ingredients and cooking steps.

Fitness Progress Database

A fitness tracking tool allowing individuals to log workouts, monitor progress, and set fitness goals.

Task Manager Database

An intuitive task management system for organizing tasks, setting priorities, and monitoring progress.



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz

For Teacher Use

Create a computer program

Domain: Digital Technology

Credits: 5 (Internal)

NZQA: <https://www.nzqa.govt.nz/nqfdocs/ncea-resource/achievements/2024/as92004.pdf>

Achieved Create a computer program	Evidence	
using a suitable programming language to construct a program that performs a specified task	Task defined at the start and program achieves what it set out to do	
testing and debugging the program to ensure it works on expected cases	Testing tables complete- may be minimal and might not include boundary or invalid tests	
documenting the program with comments.	Code comments are present in throughout the code	
Merit Develop a well-structured computer program		
using succinct and descriptive variable names	Good descriptive variable names	
documenting the program with comments that clarify the purpose of code sections	Good comments that are informative	
testing and debugging the program to ensure it works on expected and boundary cases.	More complete testing tables including tests for boundary- and the program handles the boundary inputs	
Excellence Create a flexible and robust computer program		
using conditions and control structures effectively	No extra code blocks, may have used functions to make the main loop neat	
using constants, variables, or derived values in place of literals to make the program flexible	No Magic Numbers!, variables used when values are changed and constants used when needed for non changing values	
testing and debugging the program to ensure it works on expected, boundary, and invalid cases	Comprehensive testing table that tries everything to break the code and the code can not be broken.	

Comments:



Attribution-NonCommercial-ShareAlike 4.0 International ([CC BY-NC-SA 4.0](https://creativecommons.org/licenses/by-nc-sa/4.0/))

www.techquity.co.nz